

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1011	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	AKTCHAYA A	Reg. No.:	192424096
Date:		Duration:	60 minutes

Code of Conduct

I AKTCHAYA A certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Aktchaya A

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
 - Describe the major functional modules of the proposed system.
 - Identify the functional and non-functional requirements.
 - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.

Methodology/ Framework Justification	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.
--	--	--	---	---

Criteria	Mark
Problem Understanding	/5
Stakeholder & Challenge Analysis	/5
Current System Analysis	/5
Proposed Software Solution & Software Engineering Principles	/5
Methodology/ Framework Justification	/5
TOTAL	/5

UNIVERSITY ACADEMIC MANAGEMENT AND STUDENT SUCCESS PLATFORM

Problem Analysis

○ Understanding the Problem Statement

Universities handle a large number of students, faculty members, academic programs, attendance records, examinations, and learning resources. Many institutions still use separate software systems or manual processes, making academic management difficult. This results in delayed reporting, poor communication, fragmented student information, and difficulty in monitoring student performance.

The proposed University Academic Management and Student Success Platform provides a centralized digital solution that integrates all academic activities into a single platform. It helps universities manage academic operations efficiently while improving student success and institutional decision-making.

○ Objectives of the Proposed System

- Manage student admissions and academic records.
- Support online course registration.
- Monitor attendance and academic performance.
- Manage examinations and assessment records.
- Identify academically at-risk students.
- Send notifications to students, faculty, and academic advisors.
- Generate institutional and academic reports.
- Improve student engagement and learning outcomes.
- Support academic planning and decision-making.
- Ensure scalability, security, and high system availability.

● Scope of the Problem

The proposed platform will support:

- Multiple university departments.

- Undergraduate and postgraduate academic programs.
- Student admissions and enrollment.
- Course registration and timetable management.
- Attendance tracking.
- Examination and result management.
- Faculty workload management.
- Student performance analytics.
- Institutional reporting and dashboards.
- Thousands of students and faculty member.

Stakeholder Analysis

● Stakeholders and Their Roles

Stakeholder	Roles and Responsibilities
Students	Register for courses, view attendance, access results, receive notifications, monitor academic progress.
Faculty Members	Manage attendance, upload study materials, conduct assessments, enter marks, monitor student performance.
Academic Advisors	Track student progress, identify at-risk students, provide academic guidance and counseling.
University Administrators	Manage admissions, departments, academic policies, generate reports, monitor institutional performance.

Examination Cell	Schedule examinations, manage question papers, publish results, maintain examination records.
Department Heads (HODs)	Monitor departmental performance, manage faculty workload, approve academic activities.
IT Administrators	Maintain the software platform, ensure security, manage databases, perform backups and system maintenance.
Parents/Guardians (Optional)	Monitor student attendance and academic progress through authorized access.

● Challenges Faced by Stakeholders

➤ Students

- Difficulty tracking attendance and academic progress.
- Delayed access to examination results.
- Poor communication regarding academic updates.

➤ Faculty

- Manual attendance recording.
- Time-consuming assessment management.
- Difficulty identifying struggling students.

➤ Administrators

- Managing large amounts of academic data.
- Delayed report generation.
- Lack of centralized information.

➤ Academic Advisors

- Limited visibility into student performance.

- Difficulty identifying academically at-risk students.
- **IT Administrators**
 - Maintaining data security.
 - Ensuring high system availability.
 - Managing scalability for thousand of users.

Current System Analysis

○ **Existing System**

Many universities currently use separate software applications or manual methods for admissions, attendance, examinations, academic records, and communication. These disconnected systems create inefficiencies and inconsistencies.

○ **Strengths of the Existing System**

- Basic student record management.
- Department-level academic administration.
- Existing examination management.
- Digital storage of some academic records.

○ **Limitations of the Existing System**

- Fragmented academic information.
- Manual attendance management.
- Delayed academic reporting.
- Poor communication between departments.
- No centralized student performance monitoring.
- Limited data analytics.
- Difficulty identifying academically at-risk students.
- Poor integration among university systems.

○ **Problems and Gaps**

- Duplicate data entry.
- Human errors in record management.
- Slow decision-making.

- Lack of real-time academic insights.
- Limited scalability.
- Inefficient academic planning.
- Delayed intervention for weak-performing students.

Proposed Software Solution

- **Proposed Solution**

Develop a centralized University Academic Management and Student Success Platform that integrates all academic processes into a single secure system. The platform continuously monitors student performance, attendance, examinations, faculty activities, and academic records while providing real-time reports and intelligent alerts.

- **Functional Modules**

Module 1: Student Admission and Academic Record Management

- Student registration
- Academic profile management
- Document verification
- Student information management

Module 2: Course Registration and Timetable Management

- Course enrollment
- Subject allocation
- Timetable generation
- Credit management

Module 3: Attendance Management

- Digital attendance recording
- Attendance monitoring
- Attendance reports
- Low attendance alerts

Module 4: Examination and Assessment Management

- Examination scheduling
- Marks entry
- Result publication
- Grade calculation

Module 5: Student Performance Monitoring

- Performance analytics
- GPA calculation
- Progress tracking
- At-risk student identification

Module 6: Faculty Management

- Faculty profile management
- Workload allocation
- Attendance management
- Teaching activity monitoring

Module 7: Notification and Communication System

- Student notifications
- Faculty notifications
- Academic alerts
- Examination announcements

Module 8: Reporting and Analytics

- Institutional dashboards
- Department reports
- Academic performance reports •
- Decision-support analytics

- **Functional Requirements**

- Student admission management.
- Course registration.
- Attendance tracking.
- Examination management.
- Result processing.
- Student performance monitoring.
- Faculty management.
- Notification system.
- Report generation.
- Dashboard visualization.
- User authentication.
- Role-based access control.

- **Non-Functional Requirements**

- High scalability.
- High availability.
- Strong security.
- Reliability.
- Fast system performance.
- Easy usability.
- Data integrity.
- Maintainability.
- Backup and recovery.

- **Software Engineering Principles Applied**

- **Modularity**

Separate modules for admissions, attendance, examinations, reports, and faculty management.

- **Scalability**

Supports multiple campuses, departments, and thousands of users.

- **Maintainability**

Modular architecture allows easy updates and maintenance.

- **Reliability**

Accurate data processing with regular backup and error handling.

- **Usability**

User-friendly interface for students, faculty, and administrators.

- **Security**

Authentication, authorization, encrypted data storage, and role-based access control.

SDLC/Framework Justification

- **Selected SDLC Model: Agile (Scrum)**

- **Justification**

- Academic requirements frequently change.
- Features can be delivered incrementally.
- Continuous stakeholder feedback improves quality.
- Faster bug fixing and updates.
- Easy integration of new academic policies.
- Suitable for large software development teams.

- **How Agile Supports Successful Development**

- Sprint-based development enables faster delivery.
- Continuous testing improves software quality.
- Frequent reviews ensure stakeholder satisfaction.
- Easy adaptation to changing university requirements. ○

Continuous maintenance and enhancement after deployment.

Expected Outcomes and Limitations

• Expected Outcomes

- Improved academic administration.
- Better student performance monitoring.
- Increased student success rates.
- Improved faculty productivity.
- Reduced manual administrative work.
- Faster academic reporting.
- Better communication among stakeholders.
- Data-driven academic planning.
- Improved institutional transparency.
- Scalable university management system.

• How the Solution Addresses Challenges

- Centralized database removes fragmented information.
- Real-time attendance and performance monitoring.
- Automated alerts identify at-risk students.
- Integrated communication improves coordination.
- Dashboards support faster decision-making.
- Automation reduces manual workload and errors.

• Potential Risks and Limitations

- High initial implementation cost.
- User training requirements.
- Resistance to adopting new technology.
- Dependence on internet connectivity.
- Cybersecurity threats.
- Data migration challenges.
- Regular maintenance and updates required.

• **Future Enhancements**

- AI-based student performance prediction.
- Chatbot for academic support.
- Mobile application for students and faculty.
- Integration with Learning Management Systems (LMS).
- Biometric attendance integration.
- Cloud-based deployment.
- Predictive analytics for institutional planning.
- Integration with online examination platforms.
- Personalized learning recommendations.